

Trabalho Final INF1022 2026.1 - Analisador Sintatico ObsAct

Disciplina: INF1022 - Compiladores

Professores: Vitor Pinheiro e Edward Hermann

Enunciado: 1 de junho de 2026

Integrantes

Arthur Barbosa - 2310394

Gabriel Arruda - 2311723

1. Objetivo

O objetivo do trabalho foi desenvolver um analisador lexico e sintatico para a linguagem ObsAct, capaz de receber um programa escrito em ObsAct e gerar um programa equivalente em outra linguagem. A linguagem alvo escolhida foi Python.

O analisador foi implementado com a biblioteca PLY (Python Lex-Yacc), que fornece ferramentas para construcao de analisadores lexicos e sintaticos baseados em Lex/Yacc e permite a construcao de um parser LALR(1).

2. Arquitetura da Implementacao

O projeto foi dividido em cinco modulos principais:

- lexer.py: define os tokens da linguagem ObsAct e realiza a analise lexica.
- parser.py: define a gramatica LALR(1), monta a AST e reconhece programas ObsAct.
- semantic.py: realiza validacoes semanticas, como declaracao de dispositivos, observacoes, duplicatas, limites de tamanho e associacoes entre dispositivos e observacoes.
- codegen.py: gera o codigo Python equivalente a partir da AST validada.
- obsact.py: driver principal; le o arquivo .obsact, executa lexer, parser e semantica, e escreve o arquivo .py gerado.

3. Funcionalidades Implementadas

- Declaracao de dispositivos: dispositivo: {namedevice} e dispositivo: {namedevice, observation}.
- Atribuicao via set observation = VAR.
- Atribuicao via set observation = ACTEXECUTE, por exemplo set estado_ventilador = verificar(ventilador).
- Atribuicao com contexto de dispositivo: set {namedevice, observation} = VAR.
- Valores numericos inteiros nao negativos e valores booleanos TRUE/FALSE, incluindo variacoes de maiusculas e minusculas.
- Condicionais se OBS entao CMDS e se OBS entao CMDS senao CMDS.
- Blocos com multiplos comandos dentro de condicionais e suporte a ifs aninhados.
- Condicoes compostas com && e operadores >, <, >=, <=, == e !=.
- Acoes ligar, desligar e verificar.
- Uso de verificar em condicoes, como se verificar(umidificador) == 0 entao ...
- Envio de alerta simples e com variavel.
- Envio de alerta sem parenteses, pois alguns exemplos do enunciado usam essa forma.
- Envio broadcast para multiplos dispositivos.
- Inicializacao automatica de toda observation para zero.
- Geracao das funcoes de runtime em Python: ligar, desligar, verificar e alerta.

4. Estado dos Dispositivos

O codigo Python gerado mantem um dicionario interno com o estado de cada dispositivo declarado. Ao executar ligar namedevice, o estado passa a ser 1. Ao executar desligar

namedevice, o estado passa a ser 0. Ao executar verificar(namedevice), o valor retornado corresponde ao estado atual do dispositivo.

Dessa forma, apos executar ligar lampada, a chamada verificar(lampada) retorna 1; apos executar desligar lampada, retorna 0.

5. Validacoes Semanticas

- Todo dispositivo usado deve ter sido declarado.
 - Toda observacao usada deve ter sido declarada ou criada por atribuicao de retorno de acao.
 - Dispositivos duplicados sao rejeitados.
 - namedevice deve conter somente letras e ter no maximo 100 caracteres.
 - msg deve ter no maximo 100 caracteres.
 - observation deve comecar com letra e pode conter letras, numeros ou sublinhado.
 - set {namedevice, observation} valida se a observation pertence ao dispositivo informado.
- A aceitacao de sublinhado em nomes de observacao foi incluida para cobrir exemplos do proprio enunciado, como estado_ventilador.

6. Gramatica Final Utilizada

```
programa      -> devices cmds
devices       -> device devices | device
device        -> dispositivo device_open IDENT }
               | dispositivo device_open IDENT , IDENT }
device_open   -> : { | {
cmds          -> full_cmd cmds | full_cmd
full_cmd      -> obsact . | obsact | simple_cmd . | simple_cmd
simple_cmd     -> attrib | act
attrib        -> set IDENT = var
               | set IDENT = actexecute
               | set { IDENT , IDENT } = var
var           -> NUMBER | TRUE | FALSE
obsact        -> se obs entao if_cmds
               | se obs entao if_cmds senao if_cmds
if_cmds       -> simple_cmd . if_cmds | simple_cmd . | simple_cmd
               | obsact . if_cmds | obsact . | obsact if_cmds | obsact
obs           -> IDENT oplogic var | IDENT oplogic var && obs
               | verificar ( IDENT ) oplogic var
               | verificar ( IDENT ) oplogic var && obs
               | verificar IDENT oplogic var
               | verificar IDENT oplogic var && obs
oplogic       -> > | < | >= | <= | == | !=
act           -> actexecute
               | enviar alerta alert_args IDENT
               | enviar alerta alert_args para todos : namelist
actexecute    -> ligar IDENT | desligar IDENT
               | verificar ( IDENT ) | verificar IDENT
alert_args    -> ( STRING ) | ( STRING , IDENT ) | STRING | STRING , IDENT
namelist      -> IDENT | IDENT , namelist
```

7. Alteracoes em Relacao ao Enunciado

- A gramatica foi fatorada e organizada para funcionar corretamente com PLY e LALR(1).
- O corpo do if foi implementado com o nao-terminal if_cmds para permitir multiplos comandos e ifs aninhados.
- verificar foi implementado principalmente na forma verificar(dev), usada nos exemplos, e tambem aceita verificar dev.

- Foram aceitos alertas sem parenteses porque alguns exemplos usam enviar alerta "msg" Monitor.
- Foi aceita declaracao dispositivo {Monitor}, sem dois-pontos, porque aparece nos exemplos.
- Foram aceitos comandos simples sem ponto final, pois alguns exemplos finais do enunciado aparecem sem ponto.
- Foi aceito sublinhado em observacoes para nomes como estado_ventilador.

8. Testes Utilizados

Foram usados cinco arquivos .obsact na pasta testes/:

- exemplo1.obsact: declaracao de dispositivos, set, condicional simples e ligar.
- exemplo2.obsact: verificar em atribuicao, if com multiplos comandos e if aninhado.
- exemplo3.obsact: if-else, alerta sem parenteses, ligar e desligar.
- exemplo4.obsact: set {dev, obs}, verificar em condicao, if aninhado e broadcast.
- exemplo5.obsact: broadcast com variavel para multiplos dispositivos.

Tambem foram testados casos adicionais: comandos sem ponto final, verificar apos ligar, rejeicao de namedevice com mais de 100 caracteres e rejeicao de set {dev, obs} quando a observacao nao pertence ao dispositivo.

9. Como Executar

```
pip install ply
python obsact.py entrada.obsact saida.py
python saida.py
```

Exemplo:

```
python obsact.py testes/exemplo1.obsact testes/exemplo1.py
python testes/exemplo1.py
```

10. O Que Funciona

Todas as funcionalidades exigidas pelo enunciado foram implementadas e testadas: analise lexica, analise sintatica LALR(1), geracao de codigo Python, declaracao de dispositivos, atribuicoes, acoes, verificacao de estado, alertas simples e com variavel, broadcast, condicionais com multiplos comandos, if-else, ifs aninhados e validacoes semanticas.

11. Limitacoes Conhecidas

Nao ha limitacoes conhecidas dentro do escopo solicitado pelo enunciado.

12. Arquivos para Entrega

Recomenda-se entregar o relatorio em PDF, o codigo fonte do analisador, a pasta testes/ com os arquivos .obsact usados, o README com instrucoes de execucao e, opcionalmente, os arquivos .py gerados a partir dos testes. Uma forma pratica de entrega e compactar a pasta do projeto contendo codigo, relatorio e testes.